

- [JoshApp - React Native Mobile Application](#)
 - [Project Overview](#)
 - [Table of Contents](#)
 - [Setup and Installation](#)
 - [Prerequisites](#)
 - [Development Environment Setup](#)
 - [Project Structure](#)
 - [Core Features](#)
 - [1. Home Screen](#)
 - [2. Members Screen](#)
 - [3. Profile Screen](#)
 - [4. Authentication Screens](#)
 - [Authentication System](#)
 - [AuthContext Implementation](#)
 - [Protected Routes](#)
 - [Data Management](#)
 - [Local Storage Implementation](#)
 - [DatabaseService](#)
 - [API Integration](#)
 - [Product Data API](#)
 - [API Implementation Details](#)
 - [Navigation](#)
 - [Navigation Structure](#)
 - [Route Protection](#)
 - [Building for Production](#)
 - [Android APK](#)
 - [iOS IPA \(Mac only\)](#)
 - [Troubleshooting](#)
 - [Common Issues](#)

JoshApp - React Native Mobile Application

Project Overview

JoshApp is a React Native mobile application built with Expo. The app provides user authentication, product browsing, member profile management, and a personalized user profile system. This documentation covers everything from setup to deployment, with detailed explanations of the application architecture and components.

Table of Contents

1. [Setup and Installation](#)
2. [Project Structure](#)
3. [Core Features](#)
4. [Authentication System](#)
5. [Data Management](#)
6. [API Integration](#)
7. [Navigation](#)
8. [Building for Production](#)
9. [Troubleshooting](#)

Setup and Installation

Prerequisites

- Node.js (v16 or higher)
- npm or yarn
- Expo CLI (`npm install -g expo-cli`)
- Android Studio (for Android development)
- Xcode (for iOS development, Mac only)

Development Environment Setup

1. Clone the repository:

```
git clone <repository-url>
cd JoshApp
```

2. Install dependencies:

```
npm install
# or
yarn install
```

3. Start the development server:

```
npx expo start
```

4. Run on specific platforms:

```
# For Android
npx expo start --android

# For iOS
npx expo start --ios

# For web
npx expo start --web
```

Project Structure

```
JoshApp/
├── app/                                # Main application screens
│   ├── (tabs)/                        # Tab navigation screens
│   │   ├── index.tsx                 # Home screen (Products)
│   │   ├── members.tsx              # Members screen
│   │   └── profile.tsx              # Profile screen
│   ├── auth/                         # Authentication screens
│   │   ├── login.tsx                # Login screen
│   │   └── signup.tsx               # Signup screen
│   └── profile/                      # Profile-related screens
│       └── update.tsx                # Update profile screen
├── assets/                           # Static assets (images, fonts)
├── components/                       # Reusable UI components
├── constants/                       # Application constants
│   └── Colors.ts                     # Color definitions
├── contexts/                         # React contexts
│   └── AuthContext.tsx              # Authentication context
├── services/                         # Service modules
│   ├── ApiService.ts                # API integration service
│   └── DatabaseService.ts           # Local database service
```

```
| app.json  
| package.json
```

```
# Expo configuration  
# Project dependencies
```

Core Features

1. Home Screen

The Home screen displays products fetched from the DummyJSON API. Features include:

- Product cards with thumbnail, title, price, and rating
- Pagination with "Next" button to load more products
- Error handling for API failures
- Graceful degradation to mock data when offline

2. Members Screen

The Members screen shows all registered users in the application. Features include:

- Member cards with name, email, phone, and location
- Pull-to-refresh functionality
- Dynamic data fetching from AsyncStorage

3. Profile Screen

The Profile screen displays the current user's information and provides account management options. Features include:

- User information display (name, email, phone, location)
- Update profile functionality
- Delete account option
- Logout capability

4. Authentication Screens

- **Login:** Email and password authentication

- **Signup:** New user registration with optional fields

Authentication System

The authentication system is built using React Context API, which provides a centralized way to manage user authentication state across the application.

AuthContext Implementation

The `AuthContext` (in `contexts/AuthContext.tsx`) provides the following functions:

- `register`: Creates a new user account
- `login`: Authenticates a user and stores their session
- `logout`: Terminates the user session
- `updateAccount`: Updates user information
- `deleteAccount`: Removes a user account

Authentication state is persisted using AsyncStorage, allowing the app to remember logged-in users between sessions.

Protected Routes

Certain screens like Profile and Update Profile check for authentication status and redirect unauthenticated users to the login screen.

Data Management

Local Storage Implementation

The application uses AsyncStorage for persistent data storage on the device. This approach ensures data persistence across app restarts and device reboots.

Key features of the data management system:

1. **User Data Storage:** All user accounts are stored locally
2. **Session Management:** Current user ID is stored for session persistence

3. Mock Data Initialization: Default users are created on first run

DatabaseService

The `DatabaseService` (in `services/DatabaseService.ts`) provides a comprehensive API for data operations:

- User registration
- Authentication
- Profile management
- Member listing

Initially implemented with SQLite, the service was refactored to use AsyncStorage for better cross-platform compatibility.

API Integration

Product Data API

The application integrates with the DummyJSON API to fetch product data. The integration is handled by `ApiService` (in `services/ApiService.ts`), which provides:

- Product fetching with pagination
- Error handling
- Offline support with mock data
- Connection status checking

API Implementation Details

```
async fetchProducts(page: number = 1, limit: number = 10):  
Promise<ProductsResponse> {  
  if (this.useMockData) {  
    return this.getMockProducts(page, limit);  
  }  
  
  try {  
    const skip = (page - 1) * limit;  
    const url = `https://dummyjson.com/products?limit=${limit}&skip=${skip}`;  
    const response = await fetch(url);  
  }  
}
```

```
if (!response.ok) {  
  throw new Error(`API request failed with status: ${response.status}`);  
}  
  
const data = await response.json();  
return data as ProductsResponse;  
} catch (error) {  
  console.error('Error fetching products:', error);  
  return this.getMockProducts(page, limit);  
}  
}
```

Navigation

The application uses Expo Router for navigation, which provides a file-based routing system similar to Next.js.

Navigation Structure

- **Tab Navigation:** Home, Members, Profile (`app/(tabs)/_layout.tsx`)
- **Stack Navigation:** Authentication and profile screens

Route Protection

Authentication status is checked at the component level, with unauthenticated users redirected to login screens when attempting to access protected routes.

Building for Production

Android APK

1. Create a development build configuration:

```
npx expo prebuild --platform android
```

2. Navigate to the Android directory:

```
cd android
```

3. Create a `local.properties` file with your Android SDK path:

```
sdk.dir=C:\Users\<username>\AppData\Local\Android\Sdk
```

4. Build the debug APK:

```
./gradlew assembleDebug
```

5. The APK will be available at:

```
android/app/build/outputs/apk/debug/app-debug.apk
```

iOS IPA (Mac only)

1. Create a development build configuration:

```
npx expo prebuild --platform ios
```

2. Navigate to the iOS directory and open the project in Xcode:

```
cd ios  
xed .
```

3. Build the app through Xcode for a simulator or device.

Troubleshooting

Common Issues

1. Android SDK not found

Create a `local.properties` file in the `android` directory with the correct SDK path:

```
sdk.dir=C:\Users\<username>\AppData\Local\Android\Sdk
```

2. Database Issues

If you encounter database-related errors, the app may need to reset its storage. You can clear the AsyncStorage data by uninstalling and reinstalling the app or by adding a manual reset function in development.

3. API Connection Failures

The app will automatically fall back to mock data when API connections fail. Check your internet connection and API endpoint availability if you want to use live data.

4. Missing Navigation Exports

If you see errors about missing default exports in navigation files, ensure all screen components are properly exported with `export default`.

This documentation covers all aspects of the JoshApp application. For further assistance or to report issues, please contact the development team.